UNIDAD 1: ARQUITECTURA Y PATRONES DE SOFTWARE

Resultado de Aprendizaje: Arquitectura en Capas y Patrones de Diseño

Integración de Metodología Ágil, Framework Django REST, Capa de Servicios y Patrones de Diseño Gang of Four (GoF)

Proyecto:	**Django REST API Tienda Online**
Framework Base:	**Django 5.2 / Django REST Framework**
Autenticación y Seguridad:	**JWT (djangorestframework-simplejwt)**
Patrones Implementados:	**Factory Method (Creacional), Facade (Estructural), Strategy (Comportamiento)**
Suite de Pruebas:	**8 / 8 Tests Pasando (100% OK)**

1 Identificación del Modelo de Arquitectura

El proyecto Django implementa una **Arquitectura en Capas (Layered Architecture)** construida sobre el patrón fundamental **MVT (Model-View-Template / Model-View-Controller en Django REST)**, al cual se le ha incorporado una **Capa de Servicios (Service Layer)** para encapsular la lógica de negocio y desacoplarla de los controladores HTTP.

Esta arquitectura establece límites bien definidos entre responsabilidades:

- **Capa de Enrutamiento (`urls.py`):** Punto de entrada que recibe las solicitudes HTTP entrantes y las redirige hacia la vista o controlador correspondiente.
- **Capa de Vistas / Fachada (`views.py`):** Valida la autenticación mediante JWT, extrae parámetros HTTP, actúa como punto de acceso simplificado (patrón **Facade**) delegando el trabajo pesado a la capa de servicios, y serializa la respuesta JSON.
- **Capa de Servicios (`services.py`):** Nueva capa donde reside la lógica de negocio pura (algoritmos de descuento según categoría mediante el patrón **Strategy**), independiente de la base de datos o de la interfaz HTTP.
- **Capa de Modelos / Persistencia (`models.py`):** Define las entidades del dominio, las relaciones en base de datos relacional y proporciona métodos de creación controlada mediante el patrón **Factory Method**.

DIAGRAMA DE ARQUITECTURA EN CAPAS Y FLUJO DE UNA PETICIÓN HTTP



Flujo desacoplado: Petición entrante → Enrutamiento → Fachada orquestadora → Estrategia de cálculo → Persistencia ORM → Respuesta JSON.

2 Capa de Servicios y Patrón de Comportamiento: Strategy

Se creó el archivo `products/services.py` donde se aloja la lógica de negocio pura. Se implementó el patrón de comportamiento **Strategy**, el cual permite definir una familia de algoritmos de descuento, encapsular cada uno en su propia clase y hacerlos intercambiables según la categoría del producto:

products/services.py — Patrón Strategy Python

```python
# products/services.py

class EstrategiaDescuento:
    def calcular(self, precio):
        raise NotImplementedError


class DescuentoElectronica(EstrategiaDescuento):
    def calcular(self, precio):
        return precio * 0.90  # 10% de descuento


class DescuentoRopa(EstrategiaDescuento):
    def calcular(self, precio):
        return precio * 0.80  # 20% de descuento


class SinDescuento(EstrategiaDescuento):
    def calcular(self, precio):
        return precio         # 0% de descuento


def obtener_estrategia(categoria_nombre):
    estrategias = {
        'Electronica': DescuentoElectronica(),
        'Electrónica': DescuentoElectronica(),
        'Ropa': DescuentoRopa(),
    }
    return estrategias.get(categoria_nombre, SinDescuento())
```

Beneficio Arquitectónico (Principio Open/Closed):

Para incorporar nuevas reglas o promociones en el futuro (ej. *"Hogar"* o *"Deportes"*), basta con agregar una nueva subclase en `services.py` sin modificar las vistas ni los modelos existentes.

3 Patrón Creacional: Factory Method en models.py

En `products/models.py` se incorporó a la clase `Product` el método de clase `crear_con_categoria_default`. Este método implementa el patrón **Factory Method**, encapsulando y estandarizando la instanciación de productos garantizando una categoría predeterminada (*'Sin categoría'*) si no se especifica una:

products/models.py — Patrón Factory Method — Fragmento de Product

```python
# products/models.py (extracto dentro del modelo Product)

    @classmethod
    def crear_con_categoria_default(cls, name, description, price):
        categoria_default, _ = Category.objects.get_or_create(name='Sin categoría')
        return cls.objects.create(
            name=name,
            description=description,
            price=price,
            category=categoria_default
        )
```

4 Patrón Estructural: Facade (Fachada) en views.py y urls.py

En `products/views.py` se implementó la función `precio_final` con el decorador `@api_view(['GET'])` y protección JWT mediante `@permission_classes([IsAuthenticated])`. Esta vista actúa como una **Fachada (Facade)** al unificar y ocultar la complejidad de la consulta al ORM, la resolución de la estrategia de descuento y el redondeo monetario en una única interfaz simple:

products/views.py — Patrón Facade — DRF Function-Based View

```python
from rest_framework.decorators import api_view, permission_classes
from rest_framework.permissions import IsAuthenticated
from rest_framework.response import Response
from .models import Product
from .services import obtener_estrategia

@api_view(['GET'])
@permission_classes([IsAuthenticated])
def precio_final(request, pk):
    producto = Product.objects.get(pk=pk)
    estrategia = obtener_estrategia(producto.category.name)
    precio_con_descuento = estrategia.calcular(float(producto.price))
    return Response({
        'producto': producto.name,
        'precio_original': producto.price,
        'precio_final': round(precio_con_descuento, 2)
    })
```

products/urls.py — Registro de la ruta — Enrutador

```python
from django.urls import path, include
from .views import CategoryViewSet, ProductViewSet, precio_final
from rest_framework.routers import DefaultRouter

router = DefaultRouter()
router.register('products', ProductViewSet, basename='products')
router.register('categories', CategoryViewSet, basename='categories')

urlpatterns = [
    path('', include(router.urls)),
    path('products/<int:pk>/precio-final/', precio_final, name='precio-final'),
]
```

5 Pruebas del Endpoint en Funcionamiento con JWT

Se probó el endpoint `/api/v1/products/<id>/precio-final/` con peticiones HTTP protegidas con el token JWT obtenido del endpoint `/api/token/`, evaluando dos productos de categorías distintas para evidenciar el descuento diferenciado:

Prueba 1: Producto Categoría 'Electrónica' (Descuento 10%) — HTTP 200 OK

```
GET /api/v1/products/1/precio-final/ HTTP/1.1 Authorization: Bearer
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9... HTTP/1.1 200 OK Content-Type: application/json { "producto":
"Laptop Gamer", "precio_original": 1499.99, "precio_final": 1349.99 /* 10% de descuento aplicado:
1499.99 * 0.90 = 1349.99 */ }
```

Prueba 2: Producto Categoría 'Ropa' (Descuento 20%) — HTTP 200 OK

```
GET /api/v1/products/2/precio-final/ HTTP/1.1 Authorization: Bearer
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9... HTTP/1.1 200 OK Content-Type: application/json { "producto":
"Camiseta Deportiva", "precio_original": 29.99, "precio_final": 23.99 /* 20% de descuento aplicado:
29.99 * 0.80 = 23.99 */ }
```

Prueba 3: Validación de Seguridad sin Token JWT — HTTP 401 Unauthorized

```
GET /api/v1/products/1/precio-final/ HTTP/1.1 (Sin cabecera de autenticación Authorization: Bearer)
HTTP/1.1 401 Unauthorized Content-Type: application/json { "detail": "Authentication credentials were
not provided." }
```

6 Pruebas Unitarias del Endpoint (products/tests.py)

En `products/tests.py` se agregaron las dos pruebas unitarias a la clase `ProductTestCase`:

products/tests.py — Pruebas Unitarias para precio-final — ProductTestCase

```python
# products/tests.py (incorporado a ProductTestCase)

    def test_precio_final_autenticado(self):
        # Prueba que un usuario autenticado reciba status 200 y el campo precio_final
        response = self.client.get(f'/api/v1/products/{self.product.id}/precio-final/')
        self.assertEqual(response.status_code, 200)
        self.assertIn('precio_final', response.data)

    def test_precio_final_sin_autenticar(self):
        # Prueba que una peticion sin token sea rechazada con 401 Unauthorized
        self.client.force_authenticate(user=None)
        response = self.client.get(f'/api/v1/products/{self.product.id}/precio-final/')
        self.assertEqual(response.status_code, 401)
```

7 Ejecución de Pruebas en Terminal (python manage.py test)

Se ejecutó la suite completa de pruebas mediante `python manage.py test -v 2`. Las 8 pruebas unitarias (6 de la Práctica 5 más las 2 nuevas de la Capa de Servicios) pasaron exitosamente:

PowerShell — Suite de Pruebas Automatizadas — OK (8/8 TESTS PASSED)

```
PS C:\Users\MAAIC\OneDrive\Documentos\django> .\venv\Scripts\python.exe manage.py test -v 2 Creating
test database for alias 'default' ('test_django_api')... Found 8 test(s). Operations to perform:
Synchronize unmigrated apps: messages, rest_framework, rest_framework_simplejwt, staticfiles Apply all
migrations: admin, auth, contenttypes, products, sessions Running migrations: Applying
contenttypes.0001_initial... OK Applying auth.0001_initial... OK Applying products.0001_initial... OK
Applying products.0002_alter_product_id... OK Applying products.0003_category_product_category... OK
Applying sessions.0001_initial... OK test_create_category
(products.tests.CategoryTestCase.test_create_category) ... ok test_list_categories
(products.tests.CategoryTestCase.test_list_categories) ... ok test_create_product
(products.tests.ProductTestCase.test_create_product) ... ok test_delete_product
(products.tests.ProductTestCase.test_delete_product) ... ok test_list_products
(products.tests.ProductTestCase.test_list_products) ... ok test_precio_final_autenticado
(products.tests.ProductTestCase.test_precio_final_autenticado) ... ok test_precio_final_sin_autenticar
(products.tests.ProductTestCase.test_precio_final_sin_autenticar) ... ok test_update_product
(products.tests.ProductTestCase.test_update_product) ... ok ----------------------------------------
---------------------- Ran 8 tests in 1.295s OK Destroying test database for alias 'default'
('test_django_api')... System check identified no issues (0 silenced).
```

8 Tabla de Patrones de Diseño Identificados

A continuación se resume la clasificación formal, ubicación y justificación técnica de los tres patrones de diseño implementados en el proyecto:

Nombre del Patrón	Categoría	Dónde se implementó	¿Por qué corresponde a esa categoría?
Factory Method	Creacional	`products/models.py` Método de clase `crear_con_categoria_default`	Pertenece a la categoría creacional porque encapsula la lógica de instanciación del objeto `Product`. Garantiza la asignación o creación de la categoría por defecto (*'Sin categoría'*) sin que el cliente deba conocer los detalles de cómo se construye ni asociar manualmente la clave foránea.
Facade (Fachada)	Estructural	`products/views.py` Vista `precio_final` y ruta asociada	Pertenece a la categoría estructural porque compone y organiza múltiples subsistemas (persistencia ORM, selección de estrategia algorítmica y formateo de respuesta). Proporciona una interfaz unificada y de alto nivel que oculta la complejidad interna ante las peticiones del cliente HTTP.
Strategy (Estrategia)	Comportamiento	`products/services.py` Clases `EstrategiaDescuento` y `obtener_estrategia`	Pertenece a la categoría de comportamiento porque define una familia de algoritmos de cálculo de descuentos intercambiables en tiempo de ejecución. Permite variar dinámicamente el comportamiento de cálculo según la categoría del producto, desacoplando la lógica de negocio de los controladores.

9 Conclusiones y Beneficios Arquitectónicos

La incorporación de la **Capa de Servicios** y la aplicación de los tres patrones de diseño (**Factory Method**, **Facade** y **Strategy**) aportan ventajas sustanciales a la arquitectura de la aplicación:

- **Principio de Responsabilidad Única (SRP):** Las vistas en `views.py` únicamente se encargan de gestionar el protocolo HTTP y autenticación, mientras que la lógica de cálculo financiero reside exclusivamente en `services.py`.
- **Principio Abierto / Cerrado (OCP):** Se pueden agregar nuevas reglas comerciales creando nuevas estrategias de descuento sin necesidad de modificar el código existente en las vistas ni en los modelos.
- **Alta Cohesión y Bajo Acoplamiento:** Cada componente del sistema tiene un propósito único y se comunica a través de interfaces claras, facilitando el mantenimiento y las pruebas automatizadas.
- **Seguridad por Capas:** El acceso a la fachada está protegido por tokens JWT, garantizando que solo clientes autorizados puedan interactuar con la lógica del negocio.